

Workshop 2: Cross-validation and model selection

TECH6

Richard Foltyn
NHH Norwegian School of Economics

October 22, 2026

See GitHub repository for notebooks and data:

<https://github.com/richardfoltyn/TECH6-H26>

Exercise 1: Hyperparameter tuning via cross-validation

In Workshop 1, we evaluated polynomial approximations using a single train-test split. However, performance on a single split is subject to sample variation and depends on which specific observations are assigned to each set.

In this exercise, we revisit the same trigonometric relationship,

$$\begin{aligned}y_i &= f(x_i) + \epsilon_i \\ f(x) &= \cos\left(\frac{3}{2}\pi x\right) \\ \epsilon_i &\stackrel{\text{iid}}{\sim} N(0, \sigma_\epsilon^2)\end{aligned}$$

and use k -fold cross-validation to tune the polynomial degree systematically.

```
[1]: import matplotlib.pyplot as plt
import numpy as np
from numpy.random import default_rng
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures

# True relationship
def fcn(x):
    return np.cos(1.5 * np.pi * x)

# Recreate the demo sample (same seed as Workshop 1)
rng = default_rng(seed=1234)
N = 50
sigma = 0.2
x = rng.uniform(size=N)
epsilon = rng.normal(scale=sigma, size=N)
y = fcn(x) + epsilon
X = x[:, None]
```

Part 1 — Compute average MSE across folds

You want to find the optimal polynomial degree using cross-validation. To this end, implement the function `compute_average_mse()` using the template below.

This function takes as arguments the polynomial degree d , the sample observations (X, y) , and the number of splits `n_splits`, and returns the average training and validation MSEs evaluated across the k folds using `cross_validate()`.

You want to find the optimal polynomial degree using cross-validation. To this end, implement the function `compute_average_mse()` using the template below.

This function takes as arguments the polynomial degree d , the sample observations (X, y) , and the number of splits `n_splits`, and returns the mean squared error (MSE) for the training and validation samples averaged across all splits.

Hint: To use the MSE as the scoring metric in `cross_validate()`, pass the argument `scoring='neg_mean_squared_error'` and remember to negate the output to get the actual MSE.

```
[2]: from sklearn.model_selection import cross_validate

def compute_average_mse(d, X, y, n_splits=10):
    """
    Compute mean squared error averaged across splits in k-fold cross-validation.

    Parameters
    -----
    d : int
        Degree of polynomial.
    X : array-like
        Explanatory variable.
    y : array-like
        Response variable.
    n_splits : int
        Number of splits in k-fold cross-validation.

    Returns
    -----
    mse_train : float
        MSE on training sample averaged over all splits.
    mse_validate : float
        MSE on validation sample averaged over all splits.
    """

    # TODO: Create pipeline object

    # TODO: Run cross-validation using cross_validate()

    # TODO: Compute MSE on train and validation samples averaged over all splits

    # TODO: Return average MSE on train and validation samples
```

Part 2 — Optimal polynomial degree

Using the function you wrote, compute the average MSEs for a sequence of 16 polynomial degrees $d = 0, 1, 2, \dots, 15$ using 10 splits.

Use the MSE statistics to plot the validation curve showing the average MSE on the y -axis against d on the x -axis. Which degree d results in the lowest average MSE on the validation sample?

Part 3 — Plotting the fitted model

Re-estimate the model using the optimal polynomial degree you just found and create a scatter plot with the original data, the true function $y = f(x)$, and the fitted polynomial.

Part 4 — Automatic cross-validation

The steps in Parts 1 and 2 can be implemented more easily using `validation_curve()`. Re-implement the cross-validation using this function by completing the code template below.

Hint: Don't forget that you have to use the *negative* MSE as the relevant criterion, i.e., specify the argument `scoring='neg_mean_squared_error'` when calling `validation_curve()`.

```
[9]: from sklearn.model_selection import validation_curve

# TODO: Create estimation pipeline

# TODO: Complete the call to validation_curve()
# train_scores, validate_scores = validation_curve(
#     estimator=...,
#     X=X, y=y,
#     param_name=...,
#     param_range=...,
#     scoring=...,
#     cv=10
# )

# TODO: Average train_scores and validate_scores across CV splits

# TODO: Report the polynomial degree that minimizes the MSE on the validation sample
```

Exercise 2: Categorical features

In this exercise, we practice fitting models that include categorical features in the regressor matrix X using the Ames housing dataset. There are several categorical variables in this data that come in two different kinds:

1. **Nominal categorical variables** capture qualitative categories. For numerical purposes, we often encode these as integer values, but any particular integer representation is arbitrary. It is a serious mistake to include such integer-valued categorical variables as *continuous* features in a model.

In the Ames housing data, the Neighborhood column is one such nominal categorical variable (stored as a string containing the neighborhood name).

2. **Ordinal categorical variables** represent data that have a natural ordering, but the distance between any two values can be arbitrary.

For example, in the Ames housing data the OverallQuality variable is an ordinal categorical variable on a scale of 1–10. While a value of 2 is clearly better than a value of 1, it is up to interpretation how much better.

Such variables can be included as categorical *dummy variables* in a model, not imposing any particular interpretation on the distance between two values. Alternatively, such variables can be included as *continuous* variables, which imposes how the distance is interpreted. For example, including an ordinal categorical variable as a linear term in a model imposes that moving from 1 to 2 has the same effect on the outcome variable as moving from 2 to 3.

Part 1 — Creating the estimation sample

We load the Ames data the same way as we did in the lectures:

```
[14]: import pandas as pd

# Use local data/ folder
DATA_PATH = '../..data'

filename = f'{DATA_PATH}/ames_houses.csv'
df = pd.read_csv(filename, sep=',')
```

First, perform the following data preprocessing steps:

1. Drop all observations that have any missing values for SalePrice, OverallQuality, or Neighborhood.
2. Tabulate how many neighborhoods have fewer than 40 observations, and drop observations from these neighborhoods from the dataset.

Part 2 — Exploratory data analysis

Before fitting a model, you should perform some exploratory data analysis to familiarize yourself with the data:

1. Compute the correlations of SalePrice with all numerical variables in the data. Which variable is the most correlated?
2. Create a histogram for the variable OverallQuality, showing the number of observations for each of the possible values 1–10.
Hint: Create a Series with the number of observations per quality level and plot it as a bar chart using `Series.plot.bar()`.
3. Create a box plot showing the SalePrice by OverallQuality.
Hint: You can use the pandas plotting function `DataFrame.plot.box()` for this.
4. Create a box plot showing the SalePrice by Neighborhood.

Part 3 — Train-test split

Split the data into a training and a test sample, assigning 40% of observations to the test sample, and report the number of observations in each set.

Stratify your split by Neighborhood so that each neighborhood is approximately equally represented in the training and test samples (use the stratify argument of `train_test_split()` for this). Specify a random seed of 1234.

Note that you can pass the whole DataFrame to `train_test_split()` instead of individual X and y values. The function will then return two DataFrames, one containing the training and one containing the test data.

Part 4 — Predict sale price by neighborhood

Create an estimator pipeline to fit the model

$$\text{SalePrice}_{ij} = \mu_j + \epsilon_{ij}$$

where i indexes observations and j indexes neighborhoods. That is, the sale price for a house i in neighborhood j is predicted to be the average sale price μ_j in that neighborhood.

In order to estimate this model, you'll have to convert the categorical string variable `Neighborhood` to a set of 0/1 dummy variables. This can be achieved using the `OneHotEncoder` as a step in a pipeline.

Fit the model on the training data, and report the RMSE both on the training and the test sample.

Part 5 — Predict price with polynomial in overall quality

You are now asked to fit the following model:

$$\text{SalePrice}_i = \beta_0 + \beta_1 \text{OverallQuality} + \beta_2 \text{OverallQuality}^2 + \dots + \beta_K \text{OverallQuality}^K + \epsilon_i$$

where the sale price is assumed to be a function of a polynomial in overall quality where we vary the polynomial degree K . We are thus implicitly treating the ordinal categorical variable `OverallQuality` as a continuous variable, which may or may not be admissible depending on the data. We will compare the estimation results to treating `OverallQuality` as a proper categorical variable in the next part.

Use the following template code to fit the above model for polynomial degrees $d = 1, 2, \dots, 9$ and compute the cross-validated RMSE on the training and validation samples. For this you should use the function `cross_validate()`. This function returns a dictionary with the keys `'train_score'` and `'test_score'` that contain the computed scores for each split. Note that you need to pass `return_train_score=True`, otherwise the function does not return the scores for the training data.

Create a plot showing the validation curves for both the training and validation samples, and report the polynomial degree that minimizes the RMSE on the validation sample.

```
[29]: import numpy as np

degrees = np.arange(1, 10)

rmse_train_mean = []
rmse_validate_mean = []

for d in degrees:
    # TODO: Create polynomial features + estimation pipeline

    # TODO: Compute RMSE using cross-validation
    # scores = cross_validate(
    #     estimator=...,
    #     X=X_train, y=y_train,
    #     scoring=...,
    #     cv=5,
    #     return_train_score=True,
    #     n_jobs=-1
    # )

    # TODO: Compute average RMSE over all CV splits
    # rmse_train_mean.append(np.mean(-scores['train_score']))
    # rmse_validate_mean.append(np.mean(-scores['test_score']))

pass
```

Part 6 — Predict price with quality dummies

You are now interested in whether treating the overall quality as a proper categorical variable improves the model performance. To this end, estimate the model

$$\text{SalePrice}_{ij} = \mu_j + \epsilon_{ij}$$

where i indexes observations and j indexes the overall quality categories 1–10. That is, the sale price for a house i with overall quality j is predicted to be the average sale price μ_j of all houses with that overall quality.

Create an estimation pipeline to fit the model and perform cross-validation exactly as you did for the polynomial case above. Note that cross-validation is not needed for this model at all since it has no hyperparameters, but doing so creates a fair comparison to the polynomial case since we are estimating and predicting on the same sample using the same splits.

Recreate the validation curve from above and add two horizontal lines showing the average RMSE from the dummy variable model for the training and validation sets.

```
[34]: # TODO: Create pipeline with dummy variables + estimation

# TODO: Run cross-validation
# scores = cross_validate(
#     estimator=...,
#     X=X_train, y=y_train,
#     scoring=...,
#     cv=5,
#     return_train_score=True,
#     n_jobs=-1
# )

# TODO: Compute average RMSE over all CV splits
# rmse_train_dummies_mean = np.mean(-scores['train_score'])
# rmse_validate_dummies_mean = np.mean(-scores['test_score'])
```

Part 7 — Compare test sample predictions

Finally, use the best polynomial model and the dummy variable model estimated on the whole training sample, and compute the RMSE on the test sample. Which model performs better?